

ABRA — the plain-English deck

Version 3.59.0 • 2026-08-06 • Will Hooper

A slide-by-slide, jargon-light tour. The white paper (linked on the last slide) has the math and sources.

Slide 1 — ABRA

A family of small AI models for competitive Pokémon (Champions VGC). Built from thousands of real ladder games. Honest about what it can and can't know.

Slide 2 — The big idea

You **can't reliably predict who wins** a match from the two team sheets — in this format it's close to a coin flip, and even a strong player-rating model barely beats a coin. So ABRA stops trying to call the winner and instead **helps you make better decisions**: what to bring, and what to do turn by turn.

This is the same move sports analytics made with "expected goals" — you can't predict the final score, but you can measure the value of each shot. ABRA is that idea, pointed at Pokémon.

Slide 3 — How it's built

One rule: **keep every real game, and analyse on top of it**. ABRA saves every public ladder replay into a growing library and builds its models on that library — so it gets smarter as more games come in, and never has to re-download anything. It runs on a normal laptop; no special hardware.

Slide 4 — The town of models

Each model is a "house" you can visit on the site:

- **MEDICHAM** — the damage engine. Matches the community standard, but disagrees with the game's OFFICIAL engine by a wide margin, so it is being replaced by that engine and kept only as a lookup.
 - **MEW** — plays the official engine against itself, so we are not limited to the games people happen to upload.
 - **GURU** — reads the metagame: which team styles beat which, from real results, with error bars.
 - **XATU** — reads the opponent: the likely item, ability, and moves behind each Pokémon.
 - **PORY** — mid-battle win chance. **Retracted as a finding**: it works out to counting how many Pokémon each side has left and how healthy they are, and it does not beat exactly that baseline.
 - **The scoring bot** — the part that finally *looks at the board*.
 - **SLOWKING** — the strategist: there's no single best team, so it plays a smart mix.
 - **CHOMP** — the team picker: which four to bring, which two to lead.
 - **ALAKAZAM** — the coach (in progress): the best move to make, right now.
-

Slide 5 — The win: the practice opponent can finally see

Everything ABRA claims about "this build beats that build" comes from playing the game out against a practice opponent. That opponent used to pick its move purely by **how popular the move is** — it had no idea what was standing in front of it. So it fired attacks at Pokémon that were flat immune, it used moves that couldn't possibly work, and when there were two enemies to aim at, it picked one by flipping a coin.

We fixed it by **watching what people actually do**. We took about 48,000 real decisions from games where both players' full teams were public — so we could see not just what someone clicked, but everything they *could* have clicked — and learned how much a real player's choice depends on what's in front of them. Nobody wrote the rules; they were measured.

The result, checked on games the learning never saw:

- it now hits the enemy's weakness **half again as often** as before,
- it wastes **a third fewer** moves on things that simply cannot work,
- and it almost never fires at something immune any more.

It is still short of a human on all three, and one thing it still does badly is decide **when to switch** — that's the next job. But the practice opponent is no longer playing blind, which means the numbers it produces are worth more than they were.

PORY also beats a coin flip mid-game, using how many Pokémon each side has left and their health, and it's *calibrated* — when it says 70% it's really about 70%. It's live on the site as a per-turn "you're at X%".

Slide 6 — The honest parts (this is a feature)

Two things we tested and reported straight, even though they're negatives:

- **Picking the team doesn't beat a coin.** We tested whether the team-picker's choices track who wins — they don't, more than chance. So we don't oversell it. The damage math is still exact and useful.
- **The "rock-paper-scissors" metagame is a hint, not a fact.** The data suggests Trick Room beats Hyper Offense beats Sand beats Trick Room — but each of those rests on only ~13–18 games, so the error bars are wide. We label it suggestive, and it'll firm up as more games arrive.

Added 3.32.0 — three more, and the first one is the worst thing we've found.

- **We thought Rage Powder was deleting attacks. It wasn't — and catching that is the story.** The test we wrote to check it aimed a *Dragon*-type attack at a Pokémon that is immune to Dragon. The redirection worked perfectly and moved the attack onto something that takes zero damage, so the test saw zero either way and we believed the wrong thing for several hours. Re-run against a Pokémon that can actually be hurt, the untouched code works fine. Nothing was broken and nothing we had measured was invalidated. **A test can be wrong in exactly the shape of the bug it is looking for** — that is now the ninth time here, and the first time it fooled us.
- **A real one next door, though.** Lightning Rod and Storm Drain — abilities that pull attacks in — genuinely weren't working, 1,901 uses. Fixed.
- **The number we quote for "how often we disagree with the real game" wasn't repeatable.** It picked its test matchups at random and never wrote down which ones. Two runs on identical code gave 6 and then 3. It now uses a fixed list, and the honest figure is 4 disagreements in 400.
- **The part of the bot that judges "am I winning?" is no better than a coin flip.** We had measured this before on 350 games — too few to support the claim either way. On 6,886 games it's decisive: when it

says it's 94% to win, it wins 54%; when it says 6%, it also wins 54%. Worse, the version we had been measuring wasn't the one the bot actually uses.

Added 3.33.0 — we went looking for the same mistake everywhere else, and found it twice more.

The 3.32.0 note above says a result we'd published as a pass turned out to be unreproducible. Here's *why*, because the reason is more useful than the result: the file we kept recorded the answers and not the settings. It's like keeping a stopwatch time and not writing down which race it was. Two runs with very different settings produced files that were identical, and we could not tell which one we had.

So we checked the other three tests in the same series.

- **"The new search picks a different move 73% of the time" is a headline with its control missing.** A search that's just guessing disagrees with *everything* — including with itself. So the test measures that too: run the same search twice with different dice and see how often it contradicts itself. That number is the yardstick, and the test printed it to the screen and never saved it. On an early version the yardstick was *higher* than the headline. We believe the 73% is real, because the yardstick shrinks as you give the search more thinking time and this run had plenty — but believing and having measured are different things, and it's the second one we publish.
- **"A leaf costs 5.83 milliseconds" was timing something the bot doesn't do.** The measurement quietly used the settings the code falls back to when you don't say, and the bot uses different ones — a randomised payout at triple the length. Not wrong arithmetic; the wrong thing measured.

Every one of these tests now writes a small companion file recording exactly what it did: how many rollouts, what randomness, what horizon, which version of every file it read, and whether anything was uncommitted at the time. Older results get one reconstructed from the commit that carried them, labelled as a guess rather than a record.

Added 3.34.0 — the clearest example yet of how this goes wrong.

We built a page showing every model, published it, and asked for a review. It was completely broken — one stray quotation mark meant none of it could run, so it showed a title and a blank screen.

Two automated checks looked at that page and both gave it full marks. One counted twelve model cards and found twelve. The other checked that every number on it cited a source, and scored it 100%. Neither of them ever *ran* the page — they read it the way you'd read a document. So a page that could not work passed every test we had.

A person opening the link found it in about two seconds.

The same day, a second version of the same problem: two new pages were written, tested, saved and published to the code repository — and never copied to the folder the live website actually serves. The check meant to catch that compared one specific file, so a *missing* file was invisible to it. Again: found by opening the URL.

Both checks now exist in a form that would have caught them, and both were written the same way — by asking what the check could not see, rather than what it did see.

Saying what we *can't* prove, as plainly as what we can, is the whole point.

We also caught ourselves twice this week, which matters more than any single fix. A result we'd published as a pass — "playing the position out beats just counting what's left" — turned out to be unreproducible from anything we'd kept, and on the evidence that survives it's a tie, not a win. And a claim written in our own notes at 3am about an outside dataset was wrong within four hours; it's corrected in place with the reason, not quietly deleted.

Slide 7 — What's next

The one lesson that changed the plan. We spent a day teaching the bot more about Pokémon — four separate things it didn't know before. All four made no difference at all. Then we changed *what it was trying to do*, twice, and both times it got a lot better.

Teaching it more facts had stopped helping. Changing its goal was what worked.

The reason is simple. The bot was trained to **copy what humans click**. That is a different goal from **winning**. It even shows up in the numbers: the model rates "use a spread move that hits both of them and doesn't hurt my own partner" as a *bad* idea — not because it is bad, but because humans rarely click it. A bot told to avoid its own best plays will avoid them.

- **Teach the two Pokémon to act as a team.** We already built this once. It plays worse, and now we know why: it was taught to *resemble* people rather than to *win*. Retraining it on the right goal is the single most promising thing on the list.
- **Fix a small unfairness in the training data.** When a Pokémon is locked into one move, we were still showing the bot all four as if it had a choice. It affects about 1 in 15 items.
- **Find out how readable it is.** We have no answer to this. An old figure said a rival bot built to beat ours won 63% of the time; that number is **withdrawn** — it was measured on a third of the features the bot now uses, on a dirty set of games, before most of the engine was fixed. The re-run was ruined when the thing being tested got rebuilt halfway through. So the honest position is that the question is open, not that the answer is bad. A bot can get better on average while staying just as easy to read; those are different questions and only one has been checked.
- **Then, looking ahead.** Right now the bot only thinks about *this* turn. Looking one turn further is a real project, but it's now measured rather than guessed: there are only **72** sensible combinations to consider each turn, not the trillions people assume.
- **ALAKAZAM** — the in-battle coach. A light version runs in your browser; a version that could beat top humans needs a rented cloud computer and months of the AI playing itself.

Something we got wrong, kept here on purpose. For two days nobody updated these documents while the code kept changing. The next person to read them — which was us — then described several models completely incorrectly, including calling one "not built yet" when it had been built, tested, and measured. Documents that lag the work don't just go quiet; they actively mislead.

Slide 8 — You cannot measure something while you are changing it

Four teams work on ABRA at once. That is deliberate — it is why the work was split four ways.

One night all four were running. Their files were kept separate so nobody could overwrite anybody. A 7,100-game experiment — "can anything beat our bot?" — was destroyed anyway.

The reason is simple once you see it. Halfway through the experiment, another team **improved the bot being tested**. So the first half of the experiment played against the old bot and the second half played against the new one. The final number described neither.

Nothing broke. No error appeared. The result looked completely normal, and the automatic checker approved it — because that checker was asking "*is this file newer than the thing it came from?*" and the answer was yes. A file can be newer than something it never actually read.

The fix is not to make the teams take turns. That would throw away the whole reason for having four of them.

Instead, an experiment now works from a **photograph of the bot** rather than the bot itself. Before measuring, we freeze a copy of everything that could change the answer, and the experiment reads the frozen copy. Other teams can rewrite the real thing all night; the experiment never notices.

The honest consequence: **we currently have no answer to "how beatable is our bot?"** The old answer was measured on a version of the bot that no longer exists, and the new attempt is void. Saying so is better than quoting a number we cannot stand behind — that number had been called the most important one in the project.

One thing did survive, and it is genuinely useful: when the bot plays a mirror match against itself it wins 49.7% of the time, which is a coin flip. That confirms our testing setup does not quietly favour one side — a worry an earlier, smaller sample had raised.

Slide 9 — Practising in different conditions from the match

We found something bigger than the bug we went looking for.

In Champions, players often reveal their whole team before the battle. Our bot **uses** that information when it plays. But when we *taught* the bot, we were throwing half of it away — the bot learned without knowing the opponent's abilities or moves, then plays with both.

It affects **half of every decision** it was trained on, and virtually every game. It is like training a driver in fog and then handing them the keys on a clear day: not obviously worse, but they never learned to use what they can now see.

We have not fixed it yet, and that is deliberate. Fixing it means retraining everything, and there is a real question first: sometimes opponents *decline* to reveal their team. A bot trained to depend on information that is sometimes missing can fail badly when it disappears. We have made that mistake before, in exactly this shape, and it is worth one deliberate decision rather than a quick fix.

Update (version 3.40.0). The decision was made: the bot plays with the full team sheet, always, and the rarer no-sheet games are set aside for now. The first half of the retraining is done — the move-picking layer now learns from the same information it plays with, and we *counted* that the information actually arrived during training (99.7% of decisions) instead of assuming it. The second half (the layer that picks pairs of actions) is queued next. No claim yet that the retrained bot is better — that comparison is set up against a frozen copy of the old one and runs next.

Also in this pass: the simulator learned eight more rule families and now covers 181 of the 186 mechanics we probe for, with the remaining 5 named and explained; and we compared our two separate "rulebook" files fact by fact — they disagreed exactly twice, and one of those (how often Iron Head flinches in this format) was a real bug the bot had been playing with. It reads the right number now.

Update (version 3.41.0, same night). The second half of the retraining is done too — the layer that picks pairs of actions now learns from the full team sheet, and we measured what all that sheet information is actually worth by replaying 45,000 real decisions with and without it. Honest answer: the bot judges positions measurably better with the sheet, but we cannot yet show it clicking different moves often enough to prove a win-rate gain — the effect is smaller than our measurement noise. We say it exactly that way rather than rounding up. Meanwhile the simulator's coverage grew again (202 of 205 probed mechanics, 3 remaining, each with a written reason), the mirror-match test hit 100% agreement with the official engine, and two real bugs — one in how Intimidate interacts with certain abilities, one where Sheer Force was hurting its own holder — were found and fixed with proof.

Update (version 3.42.0) — we were teaching the bot moves nobody ever picked.

Will said it first: *"i def dont like just tossing turns because they got outplayed with a move like Encore or Follow Me, these are the basis of VGC."* He was right, and it turned out to be worse than tossing them.

When Encore hits you, the game **takes the controls away** and makes you repeat your last move. The replay just shows you using that move. Because it is a move you legitimately own, every check we had said "looks like a normal choice" and the model dutifully learned it as one. The same thing happened whenever Roar or Whirlwind blew a Pokémon in — the replay writes that arrival exactly like a voluntary switch, so we were teaching the bot to make switches the player never made. **1,336 of 241,927 recorded actions across 8,942 games were things no human chose.** That is not a small rounding error being fed to the model; it is a small amount of outright fiction, and it was concentrated on the turns where the opponent had just outplayed the player. Those are gone now, and counted, so we can see the number.

The other half — Follow Me and Rage Powder — is subtler. The replay only records where an attack *landed*, so when a Pokémon soaks the hit we could not tell which target the player was aiming at. We used to write down "they aimed at the soaker", confidently, which was usually false. Now the model is told the honest thing: it was one of these two, we do not know which, learn accordingly. There is a standard statistical method for exactly this, and we tested it first on data where we had hidden the right answer ourselves — it recovers 97% of the damage when the problem is severe.

And here is the part we are not going to round up. Removing the fiction worked: on those turns the model now puts measurably less confidence in the move nobody picked. The Follow Me fix bought **nothing we can measure** — and our own test had predicted that before we ran it, because those turns are only about 1.3% of the data and the ambiguity is only ever between two targets. Overall accuracy did not move at all. We are doing it anyway, because a wrong label is a wrong label and the model has no way to tell us which of its mistakes came from one.

And then the simulator changed underneath both of those numbers, so we checked them again (3.47.0). A measurement is only worth the version of the code it was taken on. Two of these results had been computed with a simulator that changed the same day, which normally means the results have to be thrown away and re-earned. Before doing that we asked the cheaper question: did the change actually alter any of the 58 things the model looks at? We ran every one of them through the old simulator and the new one, on 1.75 million real decisions, and got byte-for-byte the same answers — so the change was real in the game engine and invisible to the model. We re-ran both measurements anyway, on a corpus that had grown by nearly three hundred games, and every conclusion above came back the same.

One more thing we found and did **not** fix: when a Pokémon is Taunted or Encored its menu shrinks, sometimes to a single legal option, and we were still scoring the player as though they had picked that move out of nine. That is about 1.6% of actions. It is a real problem, it is a different problem, and fixing it means retraining everything again — so it is written down with a number attached rather than quietly bundled in here.

Slide 9b — The rule was there. It was just pointed at everybody. (3.44.0)

Psychic Terrain is a floor that stops fast moves — Fake Out, Sucker Punch, Extreme Speed, the moves that go first. Our simulator knew that. What it did not know is that **the floor only protects Pokémon standing on it.** A Talonflame is in the air. A Pokémon with Levitate is in the air. Fake Out hits them straight through the terrain, and we were blocking it.

Fake Out is the single most-clicked move in this format — nearly thirteen thousand uses in our corpus. So this is not a corner case; it is a rule the practice opponent got wrong in a large slice of the games it plays.

Two things about it are worth more than the fix.

The first: the question *"is this Pokémon on the ground?"* was already answered in three separate places in the same file, by hand, and **the three answers did not agree with each other**. One of them was even keeping a counter of how often it knew it was giving the wrong answer. That counter had been running since the previous release, next to a note saying the information needed to fix it was not available — and the information had arrived a release earlier. There is now one function, and the four places that need the answer all ask it.

The second: our own test suite could not have caught this. The existing check for Psychic Terrain aims the blocked move at a **Garchomp**, which is standing on the ground, so it passes whether the rule is right or wrong. Every instrument we own asks *"does this mechanic happen?"*. None of them asked *"does it happen only where it should?"* — and that is the fourth bug of exactly this shape in two days. The new check has five arms, and one of them exists purely to catch a mistake we nearly made: Orthworm's Earth Eater makes it immune to Ground attacks, which looks in our data exactly like Levitate, and Orthworm is very much on the floor. We asked the official game engine and it told us so.

Slide 9b — We said we had 97 bugs. We had none of them.

A tool of ours changes a fact the simulator is supposed to read, then watches whether the simulator's behaviour moves. If nothing moves, the simulator was not really reading that fact. It flagged **97** of these, and they were passed on as 97 bugs.

Will asked what the top two actually were. **Both were fine**. Life Orb's damage boost is read from the data; only its recoil is written against the item's name, which is untidy but not wrong today. Light Screen's number is ignored **deliberately** — the data file carries the singles-format value, and this is a doubles engine where the real number is different. Using the data file's number would have made every Light Screen look a third better than it is.

The tool could see that nothing moved. It could not see **why**. It now answers that too, by reading the simulator's own source code rather than anybody's comments about it, and sorts every finding into four boxes: nothing implements this at all (the real bugs), the engine substitutes its own value on purpose, the engine works off the name instead of the data, and the test simply never reached that branch. **None of the 97 is in the first box**. The count we now track is that first box only — 163 items, none of them from the 97 — because a number that includes false alarms is a number people stop reading.

The rule was checked against three cases we had already worked out by hand, and it refuses to publish anything if it cannot reproduce all three.

Slide 9c — Everything that changes accuracy was switched off, and one move was charged for and never delivered (3.56.0)

Some moves miss. Some things make you miss less: Coil sharpens your aim, a Wide Lens is an item that does the same. Some things make you miss more: Sand Veil hides a Pokémon in a sandstorm. And No Guard means nobody ever misses at all. All four are in the game we are modelling, and together they show up about five thousand times in our replay store.

We measured each of them the boring way — play it out with the thing, play it out without the thing, compare. **All four returned exactly the same number both ways**. Not close, not within noise: identical, to the digit. Something that changes nothing at all is not a subtle bug. Every one of them was simply not connected.

Three separate reasons, which is the part worth noticing. A translation table turned the words "accuracy" and "evasion" into nothing, so a move that was supposed to raise three of your stats quietly raised two. Items and abilities were never consulted for accuracy anywhere. And the function that decided whether you hit **was not given the attacker or the target** — it was handed a move name and asked to answer a question that depends on who is holding what. It could not have got that right even in principle.

In the same pass: **Substitute took a quarter of your health and gave you nothing**. The doll was paid for and never built. That is 1,976 clicks in the store of a move that was strictly worse than doing nothing — the simulator would tell you the cost and then hand you an empty box.

The tempting fix for Substitute was the bigger bug, and we measured it before writing it. Some moves go *through* a Substitute — sound-based ones do, famously. It would have been natural to write the rule as "sound gets through". We checked: the three moves that most often need to get through in this format are Encore, Taunt and Disable, together about seven thousand uses, and **not one of them is a sound move**. A tidy rule would have walled all three.

Coverage is now 231 of 232 probed mechanics. The one that remains has a written reason, as do five more rules we know are absent — the largest of those needs information the damage function is never handed, so wiring it is a design decision rather than a fix, and it is written down as one instead of being quietly patched.

Slide 9d — The simulator could not say what it did, only where it ended up (3.58.0)

The problem. We check our simulator against the official one by comparing *results* — how much damage, what the board looked like after a turn. When they disagree, that tells you *that* something is wrong and almost nothing about *what*.

What the official simulator has that we did not. A running commentary. Every decision it makes it writes down, labelled: *Incineroar used Fake Out. Garchomp's Attack fell. The Focus Sash broke*. It is a transcript with the reason attached to every line.

What changed. Our simulator writes the same transcript now, in the same format, off by default so it costs nothing when nobody asks. The list of things it can write down was **read out of the official simulator's source code**, not typed from memory — and it checks itself two ways: it fails if we claim to write something the real one never writes, and it fails if the real one writes something we neither write nor have an explanation for. Ninety-one kinds of event exist; we write thirty-six, and the other fifty-eight each have a sentence saying why not.

What it caught on the first night. Two things, and both are about our own measuring tools rather than about the game:

- Our damage test only ever compares the *lowest* and *highest* possible rolls. It never looked at the fourteen in between — and in between, the two simulators do not agree about how likely each one is. We had been reporting 149 out of 150 correct on a comparison that could not see the middle.
- Our simulator does things in a different *order* inside a single attack than the real one does. The board ends up identical, which is exactly why the old test said everything was fine.

Neither is fixed yet, on purpose. Changing how a damage roll is picked would move every measurement this project has ever recorded. The point of tonight was to build the instrument that can see it.

Slide 9e — Somebody built a bot that beats a pro. Any strong human learns to beat it in five games. (3.59.0)

The result. A university team (Angliss, Cui, Hu, Rahman and Stone, published at AAMAS 2026) built a Pokémon doubles bot the proper modern way: show it 700,000 human games so it learns to imitate people, then let it play itself millions of times to get better. It works. **In a mirror match it beat a World Championships competitor.**

And then they measured how easy it is to beat, and the answer was: completely. They trained a second bot whose only job was to beat the first one, and it did — almost every time. Their human tester said it plainly: *the agent is strong at first, but after enough games in a row, strong players adapt and beat it.* Against a merely *advanced* player it won two games out of five.

We think that is the whole problem, not a bug in their work. Here is the thing to picture. Their bot is a **memorised answer sheet**: for every situation, one answer, worked out in advance and then frozen. That is a great way to play chess, where both players can see everything. It is a terrible way to play poker, where they can't — because once your opponent notices you always fold in a spot, you lose that spot forever.

Pokémon is poker, not chess. You cannot see which four of their six they brought, what they're holding, or their last move. Poker figured this out between 2007 and 2021, and the answer was: **don't memorise, re-think every hand.**

So our headline number changes. We used to ask "how often do we win?" We now ask "**how easily can a prepared opponent read us?**" That is a harder, more honest question, and it is the one their number answers too — so the two projects can be compared even though the bots can never actually play each other. Their score is about 100% readable. **Ours is currently unknown, and saying so is the point** — we've made our headline a number we can't produce yet rather than a number that flatters us.

Our bet — and it is a bet, not a claim. A bot that *recomputes* from scratch every turn should be harder to read than one that *recalls*. There is nothing to memorise about it. Whether that survives in a game where both players move at once, dice are rolled, and the whole thing is over in six turns, **nobody knows. That's the experiment.**

What we do if we're wrong. We use their recipe. It's published, it's open source, and it works. Being wrong here would be a real finding about Pokémon, not a failure.

Slide 9f — We had a number that justified two years of work, and it was wrong by five times (3.59.0)

The claim. We wrote our own Pokémon simulator instead of using the official one, and the reason written down in every document was: **the official one is 117 times slower.** That number decided the architecture.

We measured it again. It's about 25 times slower, not 117.

Why we're telling you rather than quietly fixing it. Three things.

1. The old number stays in the documents, with a dated correction beside it, because this project does not rewrite a past conclusion as though it were never made.
2. **The decision it justified is still right** — 25 times slower is still far too slow to think during a live game. So nothing is being rebuilt.
3. **But the reason has to be a real reason.** "117 times" was doing work it hadn't earned. The honest version is now a claim you could prove us wrong about: *we wrote a fast simulator so the bot can re-think*

*every turn — so the simulator is worth building **if and only if** re-thinking actually helps.* We are about to run the test that decides it.

The part that should worry a reader most. We have three different speed measurements of our own simulator, taken two weeks apart, that disagree by a factor of ten — and **not one test noticed**. We measure win rates to three decimal places and check them against a noise floor. We had never once checked the speed of the thing everything else runs on.

Slide 10 — Read more

Full technical detail, math, and sources: [ABRA white paper](#). Also: [project summary](#) · [technical docs](#) · [model ledger](#).